2026-04-19-completion-initiative

Completion Initiative — Full Report & Phased Implementation Plan

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Drive the qBitTorrent platform to zero-debt status: every module wired, every test unskipped, 100% coverage, hardened for security + concurrency + memory safety, fully documented (text + diagrams + OpenAPI + website + video), and continuously scanned by Snyk + SonarQube — without breaking any existing functionality and in full compliance with the `.specify/memory/constitution.md` (v1.1.0).

Architecture: Preserve the Container-First two-container topology (Principle I). Additive scanning/observability services are introduced as an **opt-in, separate compose file** (`docker-compose.quality.yml`) to avoid amending the constitution. All hardening is layered behind feature flags so existing working paths never change behavior silently. TDD-gated throughout (RED → watch fail → GREEN → rebuild-reboot → commit, per CLAUDE.md).

Tech Stack: - Backend: Python 3.12, FastAPI, aiohttp, uvicorn - Plugins: qBittorrent nova3 engine contract, 48 `.py` files in `plugins/` - Frontend: Angular 19/21 standalone app, Vitest - Scanners (new): Snyk CLI, SonarQube Community (compose), Semgrep, Bandit, pip-audit, Trivy, Gitleaks - Observability (new): Prometheus + Grafana + OpenTelemetry (compose; optional) - Docs site (new): MkDocs Material (static, commit-synced) - Tests:

pytest + pytest-asyncio + pytest-cov + hypothesis + pytest-timeout + pytest-randomly + Locust (load) + Playwright (browser e2e)

Part A — Report of Unfinished / Broken / Undocumented Work

All findings are verified against the current `main` at `6ea3383`. Line numbers are from the snapshot taken on 2026-04-19.

A.1 Tests that are disabled, skipped, or excluded

#	File / location	What	Count
1	`tests/integration/test_manual_issues.py`	runtime `pytest.skip()` (merge service down)	9
2	`tests/integration/test_dashboard_automation.py`	runtime `pytest.skip()`	9
3	`tests/integration/test_login_actions.py`	runtime `pytest.skip()`	8
4	`tests/integration/test_buttons_api.py`	runtime `pytest.skip()`	8
5	`tests/integration/test_ui_quick.py`	runtime `pytest.skip()` + `--ignore=` in `run-all-tests.sh:122`	5
6	`tests/integration/test_live_containers.py`	runtime `pytest.skip()`	5
7	`tests/integration/test_iptorrents.py`	runtime `pytest.skip()`	5
8	`tests/integration/test_auth_and_links.py`	runtime `pytest.skip()`	4
9	`tests/integration/test_dashboard_rendering.py`	runtime `pytest.skip()`	3
10	`tests/integration/test_magnet_dialog.py`	runtime `pytest.skip()`	1
11	`tests/integration/test_ui_comprehensive.py`	runtime `pytest.skip()` + `--ignore=`	1
12	`tests/security/test_auth_bypass.py`	runtime `pytest.skip()`	2

#	File / location	What	Count
13	`tests/security/test_input_validation.py`	runtime `pytest.skip()`	2
14	`tests/security/test_xss_protection.py`	runtime `pytest.skip()`	1
15	`tests/security/test_csrf_protection.py`	runtime `pytest.skip()`	1
16	`tests/performance/test_concurrent_search.py`	runtime `pytest.skip()`	1
17	`tests/stress/test_search_stress.py`	runtime `pytest.skip()`	1
18	`.github/workflows/test.yml:65`	`docker-check` job gated on `test_level == 'full'`	gate
19	pyproject.toml / pytest.ini	**no `--cov-fail-under` threshold anywhere**	gap

Total runtime skips: 71 across 17 test files. Root cause: environment-dependent gates (`requests.get("http://localhost:7187/").ok`) rather than fixtures that **guarantee** the services are up before running. Additionally, 2 integration files are unconditionally excluded from `run-all-tests.sh`.

A.2 Frontend tests

- `frontend/src/app/app.spec.ts` — **only 23 lines**, **one smoke test**.
- Zero tests for: `dashboard.component`, `magnet-dialog.component`, `qbit-login-dialog.component`, `toast-container.component`, `confirm-dialog.component`, `api.service`, `sse.service`, `toast.service`, `dialog.service`, `search.model`, routing, error paths.
- No Playwright/Cypress browser e2e tests.
- No coverage gate in `angular.json`.

A.3 Dead / unfinished code

#	File	Issue
1	`plugins/*.py`	**48 `.py` files** on disk; only 12 are canonical per constitution (Principle II) and `install-plugin.sh`. **36 plugins have no tests, no install path, no documentation, no health check.**

#	File	Issue
2	`download-proxy/src/ui/__init__.py`	Empty module (single comment). `ui/` serves compiled Angular assets only — the Python module itself is a no-op.
3	`plugins/socks.py:433`	`raise NotImplementedError("Received UDP packet fragment")` — UDP fragmentation unimplemented. TCP path works; document or implement.
4	`plugins/nova2.py:89`	`raise NotImplementedError` — **intentional** abstract base. Leave alone, but document.
5	`plugins/webui_compatible/{kinozal,nnmclub,rutracker}.py`	Alternate implementations per constitution Principle V — document their activation path.
6	TODO/FIXME markers	**Zero** in core code (positive finding).

A.4 Concurrency / memory / race hazards

#	File:line	Hazard
1	`download-proxy/src/main.py:69-83`	Two daemon threads spawned, `time.sleep(60)` keepalive, no graceful shutdown, no join timeout.
2	`download-proxy/src/merge_service/search.py:308`	`asyncio.gather(*[_search_one(t) for t in trackers])` — **no semaphore cap**, unbounded across ~40 trackers.
3	`search.py:233-236, 960-991`	`_active_searches` + `_last_merged_results` dicts grow without TTL/eviction — memory leak.
4	`search.py:234, 494, 622, 701, 781`	`_tracker_sessions` stores aiohttp sessions without close/rotation.
5	`search.py:378-395`	Subprocess template string injection risk (tracker name interpolated into exec'd Python).
6	`search.py:407`	10s subprocess timeout inadequate under load.
7		

#	File:line	Hazard
	`download-proxy/src/api/hooks.py:56,183-184`	`_execution_logs` global list appended by concurrent async handlers without lock.
8	`download-proxy/src/api/routes.py:395-398,418-429`	Credential JSON file writes without `flock` → corruption risk; plaintext at rest.
9	`download-proxy/src/api/streaming.py:75,123,77,181`	`seen_hashes: set()` mutated by concurrent SSE generators; `while True: await asyncio.sleep(0.5)` polls even after client disconnect.
10	`download-proxy/src/api/__init__.py:68`	`CORSMiddleware(allow_origins=["*"])` — over-permissive.
11	`download-proxy/src/api/auth.py:24`	`_pending_captchas: dict = {}` unbounded, no TTL.
12	`docker-compose.yml:12,36`	Hardcoded `admin`/`admin` default (documented as intentional by constitution III, but still needs runtime override warning).
13	Zero use of `tenacity`/`backoff`/`async-timeout`	No retry, no circuit-breaker, no exponential backoff anywhere.
14	Zero `loadChildren` / `@defer`	Angular app is fully eager-loaded.

A.5 Security / scanning tooling

Tool	Status
Snyk	**Not configured** (no `.snyk`, no workflow)
SonarQube	**Not configured** (no `sonar-project.properties`, no compose service)
Semgrep	**Not configured**
Bandit	**Not configured**
pip-audit / safety	**Not configured**
Trivy / Grype (image scan)	**Not configured**
Gitleaks / truffleHog	**Not configured**
Ruff	Config exists (`ruff.toml`) but **not in any workflow**

A.6 Documentation gaps

- **Missing READMEs** in 9 top-level directories: `config/`, `docs/`, `download-proxy/`, `plugins/`, `specs/`, `tools/`, `Upstreams/`, `tmp/`, `.specify/`.

- **No OpenAPI spec file** committed (FastAPI auto-generates, but no frozen artifact for docs/website).
- **No architecture diagrams** beyond ASCII in `README.md`.
- **No SQL schema** (project uses no relational DB — document this explicitly).
- **No docs site generator** configured.
- **`CHANGELOG.md` lags** behind recent commits (Angular 19 port, 5 dashboard fixes).
- **`PLUGINS.md`** documents 12 plugins; the other 36 on disk are undocumented.

A.7 Website

Does not exist. No `website/`, `site/`, `web/`, or docs-site generator config. `frontend/` is the dashboard app, not a marketing/docs website. **Needs user decision.**

A.8 Video courses

Do not exist. No `video/`, `courses/` directories, no mp4/webm/YouTube references anywhere. **Needs user decision — see Part B.**

A.9 Observability

- No Prometheus, Grafana, OpenTelemetry, Sentry, structured logging framework.
- Only `logging.basicConfig()` in `main.py`.
- Healthchecks exist (docker-compose.yml:21, :61) — good baseline.

A.10 CI

- `.github/workflows/test.yml` is `workflow_dispatch`-only (manual).
- No PR-triggered checks, no scheduled scans, no coverage report upload, no SARIF upload.

Part B — Ambiguities Requiring User Decision

The request mentions **extending** existing video courses and the website. Neither exists in this repo. Before Phase 9+ can begin, the user must choose one of the options below for each:

B.1 Website

- **Option 1 (recommended, cheapest)**: Generate a static docs+marketing site with **MkDocs Material** under `website/`, published via GitHub Pages. Content = rendered `docs/` + landing page + screenshots of the Angular dashboard. No runtime dependency.
- **Option 2**: Full marketing site with **Astro** + component library. More polish, more maintenance.
- **Option 3**: Defer — say "not applicable, no website in scope."

B.2 Video courses

- **Option 1**: Author **scripts + storyboards + screen-capture shotlists** for a video course series (Operator 101, Plugin Author, Contributor Deep-Dive, Security & Ops). Record them out-of-band.
- **Option 2**: Author **Asciinema cast files** (terminal sessions, captured with `asciinema rec`) plus narrated markdown — reproducible from git, zero binary bloat.
- **Option 3**: Defer — say "not applicable, no videos in scope."

The plan below assumes **Website = Option 1 (MkDocs Material)** and **Video courses = Option 2 (Asciinema + narrated markdown)**. If the user chooses differently, Phase 9 and 10 swap implementations but the phase structure is unchanged.

Part C — Test Type Catalog (what "ALL test types" means here)

This is the authoritative list of test types the plan will cover and the framework used for each. Every module must have tests in every applicable row.

#	Type	Framework	Directory
1	Unit (Python)	pytest + pytest-asyncio	`tests/unit/`
2	Unit (Frontend)	Vitest + Angular Testing Library	`frontend/src/**/*.spec.ts`
3	Integration (API)	pytest + httpx.AsyncClient	`tests/integration/`
4	Integration (Plugin ↔ nova3)	pytest + vcrpy (record/replay)	`tests/integration/plugins/`
5			`tests/e2e/`

#	Type	Framework	Directory
	End-to-end (browser)	Playwright (Python)	
6	End-to-end (API flow)	pytest + live compose	`tests/e2e/api/`
7	Contract (OpenAPI)	schemathesis	`tests/contract/`
8	Property-based	hypothesis	`tests/property/`
9	Fuzz	hypothesis + atheris (optional)	`tests/fuzz/`
10	Mutation	mutmut	runs against `download-proxy/src/`
11	Security	pytest + bandit + semgrep rules	`tests/security/`
12	Performance (latency)	pytest-benchmark	`tests/performance/`
13	Load (throughput)	Locust	`tests/load/`
14	Stress (overload)	Locust + chaos probes	`tests/stress/`
15	Chaos (fault injection)	toxiproxy + pytest	`tests/chaos/`
16	Concurrency (race)	pytest-randomly + pytest-repeat + asyncio ops	`tests/concurrency/`
17	Memory leak	tracemalloc + pytest + objgraph	`tests/memory/`
18	Deadlock / timeout	pytest-timeout (hard cap all tests)	global config
19	Smoke	pytest	`tests/smoke/`
20	Monitoring / metrics	pytest + Prometheus scrape asserts	`tests/observability/`
21	Infra / compose	pytest-docker + compose-config validator	`tests/infra/`
22	Accessibility	Playwright + axe-core	`tests/a11y/`
23	Visual regression	Playwright snapshots	`tests/visual/`
24			`tests/docs/`

#	Type	Framework	Directory
	Documentation link check	pytest + linkchecker	
25	Type-check (static)	mypy –strict + tsc –noEmit	CI job
26	Lint (static)	ruff + eslint + shellcheck + hadolint + yamllint	CI job
27	Dependency audit	pip-audit + npm audit + Snyk + Trivy	CI job
28	SAST	Semgrep + Bandit + SonarQube	CI job
29	Secret scan	Gitleaks + TruffleHog	CI job
30	License compliance	pip-licenses + license-checker	CI job

Part D — Phased Implementation Plan

Each phase ends with a **green test suite + rebuild-reboot + commit** per CLAUDE.md protocol. No phase begins until the previous phase is fully green. Each phase is an **independently mergeable milestone**.

Phase 0 — Safety Net & Baseline (prerequisite for everything)

Why first: Before any hardening, we need the current behavior locked in by tests, CI running automatically, and a rollback point. This phase **changes zero product code** — it only adds observability of current behavior.

Deliverables: - `pyproject.toml` replaces ad-hoc pytest/coverage config, centralizes tool settings. - `pytest-cov`, `pytest-timeout`, `pytest-randomly`, `pytest-xdist`, `pytest-docker`, `hypothesis`, `responses`, `freezegun` added to `tests/requirements.txt`. - `tests/conftest.py` gains **service-availability fixtures** (`merge_service_live`, `qbittorrent_live`, `webui_bridge_live`, `compose_up`) that **start the stack before the test** instead of skipping. - `.github/workflows/` split into: `syntax.yml` (push+PR), `unit.yml` (push+PR), `integration.yml` (push+PR, with compose), `nightly.yml` (full suite), `security.yml` (weekly scan). All auto-triggered. - `docker-compose.quality.yml` (additive, opt-in) added with SonarQube +

Prometheus + Grafana services, documented in `docs/QUALITY_STACK.md` and justified against Principle I. - Coverage baseline captured at commit SHA in `docs/COVERAGE_BASELINE.md`. - `.gitignore` updated for `.ruff_cache/`, `.mutmut-cache`, `htmlcov/`, `.coverage*`, `.pytest_cache/`, `prof/`.

Tasks:

Task 0.1 — Create `pyproject.toml` with unified tool config

Files: - Create: `pyproject.toml` - Modify: `tests/requirements.txt` - Modify: `.gitignore`

- [ ] **Step 1: Write test that asserts `pyproject.toml` exists and declares pytest/coverage config**

```python
# tests/unit/test_toolchain_config.py
import tomllib
from pathlib import Path

def test_pyproject_declares_pytest_and_coverage_config():
    root = Path(__file__).resolve().parents[2]
    data = tomllib.loads((root / "pyproject.toml").read_text())
    assert data["tool"]["pytest"]["ini_options"]["addopts"]
    assert data["tool"]["coverage"]["run"]["source"] ==
        ["download-proxy/src", "plugins"]
    assert data["tool"]["coverage"]["report"]["fail_under"] == 100
```

- [ ] **Step 2: Run test to verify it fails** — `python3 -m pytest tests/unit/test_toolchain_config.py -v` — expect FAIL (pyproject.toml missing).

- [ ] **Step 3: Create `pyproject.toml`** with sections `[tool.pytest.ini_options]`, `[tool.coverage.run]`, `[tool.coverage.report]`, `[tool.ruff]` (migrate from `ruff.toml`), `[tool.mypy]`. Set `addopts = "-ra --strict-markers --strict-config --cov --cov-report=term-missing --cov-report=xml --cov-report=html --timeout=30"`. Set `fail_under = 100` (start low, raise per phase).

- [ ] **Step 4: Re-run test** — expect PASS.

- [ ] **Step 5: Commit** —

```
git add pyproject.toml tests/unit/
        test_toolchain_config.py .gitignore
git commit -m "chore: consolidate tool config into
        pyproject.toml with coverage gate"
```

Task 0.2 — Add service-availability fixtures

Files: - Modify: `tests/conftest.py` - Create: `tests/fixtures/compose.py` - Create: `tests/fixtures/services.py`

- [ ] **Step 1: Write test** proving `merge_service_live` fixture starts compose when service is down and returns the healthy base URL.

```
# tests/integration/test_fixtures_bring_up_services.py
import httpx
def test_merge_service_fixture_returns_healthy_url(merge_service_live):
    r = httpx.get(f"{merge_service_live}/health", timeout=5)
    assert r.status_code == 200
```

- [ ] **Step 2: Run it — expect FAIL** (fixture doesn't exist).

- [ ] **Step 3: Implement fixture** in `tests/fixtures/services.py`. Use `pytest-docker` to bring up `docker-compose.yml` + `docker-compose.quality.yml`. Poll `/health` until 200 or timeout 60s. Tear down on session end. Register fixture in `tests/conftest.py`.

- [ ] **Step 4: Run test — expect PASS.**

- [ ] **Step 5: Commit** — `test: add service-availability fixtures that start compose stack.`

Task 0.3 — Replace every `pytest.skip("service down")` with fixture dependency

For each of the 17 files in §A.1 with runtime skips, remove the `if not service.ok: pytest.skip(...)` block and require the fixture. This converts all 71 skips into guaranteed executions.

- [ ] **Step 1: Write meta-test** that greps the test tree and asserts zero `pytest.skip(` calls remain with message `"service"` or `"available"`.

```
# tests/unit/test_no_runtime_service_skips.py
import re, pathlib
```

```
def test_no_runtime_service_skips():
    root = pathlib.Path(__file__).resolve().parents[1]
    offenders = []
    pat = re.compile(r'pytest\.skip\([^)]*(service|available|
        unreachable)', re.I)
    for p in root.rglob("test_*.py"):
        if pat.search(p.read_text()):
            offenders.append(str(p))
    assert offenders == [], offenders
```

- [ ] **Step 2: Run — expect FAIL** listing all 17 files.

- [ ] **Step 3: Edit each file** — replace `if not requests.get(...).ok: pytest.skip(...)` with `def test_foo(merge_service_live): ...`. If the test genuinely needs a CAPTCHA-shielded resource, gate with `@pytest.mark.requires_credentials` and register the mark in `pyproject.toml`; provide a `MOCK_MODE` fixture that records/replays with vcrpy.

- [ ] **Step 4: Run — expect PASS.**

- [ ] **Step 5: Remove `--ignore=` lines** from `run-all-tests.sh:122` (`test_ui_comprehensive.py`, `test_ui_quick.py`).

- [ ] **Step 6: Rebuild-reboot** (CLAUDE.md mandate): `./stop.sh && podman exec qbittorrent-proxy find / -name __pycache__ -type d -exec rm -rf {} + || true; ./start.sh -p`.

- [ ] **Step 7: Run entire suite** — `./ci.sh`. Expect PASS with 0 skips.

- [ ] **Step 8: Commit** — `test: convert 71 runtime skips into fixture-gated executions.`

Task 0.4 — Split CI into auto-triggered workflows

Files: - Modify: `.github/workflows/test.yml` → split into 5 files - Create: `.github/workflows/syntax.yml` - Create: `.github/workflows/unit.yml` - Create: `.github/workflows/integration.yml` - Create: `.github/workflows/nightly.yml` - Create: `.github/workflows/security.yml`

- [ ]

Step 1: Write test `tests/unit/test_ci_workflows.py` asserting each expected workflow file exists and has `on: [push, pull_request]` (except nightly/security).

- [ ] **Step 2: Fail** — not present.

- [ ] **Step 3: Author workflows.** Use `actions/cache` for pip + npm. Upload coverage as artifact. Use matrix for Python 3.12 + 3.13. Upload SARIF from scanners to Security tab.

- [ ] **Step 4: Pass.**

- [ ] **Step 5: Commit** — `ci: split manual workflow into auto-triggered suite`.

Task 0.5 — Coverage baseline

- [ ] **Step 1: Run** `coverage run -m pytest && coverage xml && coverage html`.
- [ ] **Step 2: Record numbers** per module in `docs/COVERAGE_BASELINE.md`.
- [ ] **Step 3: Commit** — `docs: capture coverage baseline at 6ea3383`.

Phase 1 — Security Scanning Infrastructure

Why: Plan mandates Snyk + SonarQube scanning before resolving findings. Scanning must run in containers (Compose) so it's reproducible without root/sudo — matching the user's hard constraint.

Deliverables: - `docker-compose.quality.yml` services: `sonarqube` (community), `sonar-db` (postgres), `snyk-cli` (run-to-completion container), `semgrep`, `trivy`, `gitleaks`. - `sonar-project.properties` with `sonar.sources=download-proxy/src,plugins,frontend/src`, `sonar.tests=tests,frontend/src`, `sonar.python.coverage.reportPaths=coverage.xml`. - `.snyk` ignore-and-policy file. - `scripts/scan.sh` orchestrator: runs all scanners non-interactively, aggregates reports to `artifacts/scans/<timestamp>/`. - `.github/workflows/security.yml` invokes `scripts/scan.sh` weekly + on-demand; uploads SARIF. - All scanners called with `--no-`

`interactive` / `--auth-token="${TOKEN}"` from env — **zero sudo, zero interactive prompts** (enforced by test that greps for `sudo` or `read -p` in scripts).

Task 1.1 — Add `docker-compose.quality.yml` with SonarQube + Postgres + volumes. Healthcheck-gated. Justification added to constitution addendum `docs/CONSTITUTION_ADDENDUM_QUALITY.md` referencing Principle I exception.

Task 1.2 — Add `.snyk`, `sonar-project.properties`, `.semgrep.yml`, `.gitleaks.toml`, `.trivyignore`.

Task 1.3 — Write `scripts/scan.sh` with mandatory flags `set -euo pipefail`, auto-detect runtime, never prompt. Test: `bash -n scripts/scan.sh` + `tests/unit/test_scan_script_non_interactive.py` greps for forbidden patterns (`sudo`, `read -p`, `su -`, `passwd`).

Task 1.4 — Wire scanners into `scripts/scan.sh` — each scanner runs in its own container, output JSON + SARIF, exit non-zero on high/critical findings.

Task 1.5 — CI workflow `.github/workflows/security.yml` runs `scripts/scan.sh` and uploads SARIF to GitHub Security.

Task 1.6 — Rebuild-reboot + full suite + commit.

Phase 2 — Resolve All Scanner Findings

Process (per finding): triage → write failing test reproducing the class of issue → fix → verify scan clean → commit. No finding is suppressed except with a documented `.snyk`/`sonar` waiver + expiry.

Task 2.1 — Fix **CORS wildcard** (`download-proxy/src/api/__init__.py:68`). Replace with env-driven `ALLOWED_ORIGINS` defaulting to `["http://localhost:7186","http://localhost:7187"]`. Test: `tests/security/test_cors.py` asserts `*` never in response. Regression test: existing UI still loads.

Task 2.2 — Fix **subprocess command-injection surface** (`search.py:378-395`). Replace subprocess-built Python with **direct in-process import** guarded by `importlib` + per-tracker **ProcessPoolExecutor** for isolation. Test: `tests/security/test_no_shell_injection.py` feeds `"; rm -rf /"` as tracker name and asserts `FileNotFoundError` not `CommandInjectionExecuted`.

Task 2.3 — Fix **credential file race** (`routes.py:395-398,418-429`). Add `filelock` dependency; wrap writes in `FileLock("/config/download-proxy/qbittorrent_creds.json.lock")`. Encrypt at rest with Fernet + key derived from `CREDS_KEY` env var (required on startup). Test: 50 concurrent writers, no corruption.

Task 2.4 — Fix **CAPTCHA dict unbounded** (`auth.py:24`). Replace `dict` with `cachetools.TTLCache(maxsize=1024, ttl=900)`. Test: property-based — insert 2000 entries, assert ≤1024 present.

Task 2.5 — Fix **credential leakage in logs**. Add `logging.Filter` that scrubs regex matches for `(PASSWORD|COOKIE|TOKEN)=\S+`. Test: run module with dummy creds, capture logs, assert regex-scrubbed.

Task 2.6 — Resolve all remaining Snyk / Sonar / Semgrep / Bandit findings **one by one**, each with a test. Blast the backlog in sub-batches of 5; commit per batch.

Task 2.7 — Rebuild-reboot + full suite + commit.

Phase 3 — Concurrency, Memory, and Performance Safety

Deliverables: every hazard from §A.4 has a fix + a test that would have caught it.

Task 3.1 — Bounded concurrency on tracker fan-out. Add `asyncio.Semaphore(MAX_CONCURRENT_TRACKERS=5)` (env-tunable) in `search.py`. Test: spawn 40 trackers, assert never >5 inflight via an `inflight` gauge.

Task 3.2 — TTL eviction on all in-memory dicts (`_active_searches`, `_last_merged_results`, `_tracker_sessions`, `_pending_captchas`, `_execution_logs`). Use `cachetools.TTLCache` or deque with fixed maxlen. Tests in `tests/memory/` — insert N items, await TTL, assert bounded growth via `tracemalloc`.

Task 3.3 — asyncio.Lock around `_execution_logs` and `seen_hashes`. Prove via `tests/concurrency/test_hooks_race.py` running 200 parallel writers under `pytest-randomly --count=50`.

Task 3.4 — Graceful shutdown for daemon threads in `main.py`. Install `signal.signal(SIGTERM/SIGINT, ...)` handler that sets an `asyncio.Event`, awaits tasks, joins with timeout. Test: `tests/unit/test_graceful_shutdown.py` spawns subprocess, sends SIGTERM, asserts exit ≤5s + clean stdout.

Task 3.5 — Retry/backoff/circuit-breaker. Add `tenacity` dependency. Wrap every outbound HTTP call in `@retry(stop=stop_after_attempt(3), wait=wait_exponential(1,10), retry=retry_if_exception_type(aiohttp.ClientError))`. Add `pybreaker.CircuitBreaker(fail_max=5, reset_timeout=60)` per tracker. Tests: simulate flaky endpoint, assert 3 retries then trip.

Task 3.6 — Client disconnect stops SSE polling (`streaming.py:77,181`). Replace `while True` with `async for _ in request.is_disconnected_stream()` pattern. Test: spawn SSE client, disconnect, assert generator exits in <1s.

Task 3.7 — Replace all blocking `time.sleep` inside async funcs with `asyncio.sleep`. Add `flake8-async` plugin + `ruff` rule `ASYNC101`. Test: ruff run passes.

Task 3.8 — Lazy loading. - Python: convert heavy imports in `routes.py` (`merge_service/*`) to inside-function imports where used in seldom-hit routes. Profile with `py-spy record` before/after. - Angular: convert dashboard to `loadChildren` + `@defer` blocks for non-critical panels. Measure initial bundle size before/after in `tests/frontend/test_bundle_budget.py` via angular.json `budgets`.

Task 3.9 — Semaphore around subprocess spawns in `search.py:407`.

Task 3.10 — Rebuild-reboot + full suite + commit.

Phase 4 — Dead Code Decision & Resolution

Context: 36 plugins on disk are not canonical per constitution Principle II. Options per plugin:

1. **Adopt** → add to `install-plugin.sh` managed list, write test, document in `docs/PLUGINS.md`. Requires constitution amendment (v1.1.0 → v1.2.0) to extend the canonical list.
2. **Keep as community** → move to `plugins/community/`, add `plugins/community/README.md` explaining how to install individually, add smoke test that `python3 -m py_compile` passes on each.
3. **Remove** → `git rm plugins/<name>.py` with rationale in commit.

Task 4.1 — Audit matrix. Generate `docs/PLUGIN_AUDIT.md` with a row per plugin: last modified, has test? works? action (adopt/keep/remove). Driven by a script `scripts/audit-plugins.sh` that runs `python3 -m py_compile` on each and exercises `search("ubuntu")` in isolated container.

Task 4.2 — Execute decisions per audit. Commit per group of 5.

Task 4.3 — `plugins/socks.py:433` UDP fragmentation. Decision: document as unsupported (TCP works) OR implement per RFC 1928 §7. Default: document + test that calling it raises a clear `NotImplementedError` with a link to the issue tracker.

Task 4.4 — `download-proxy/src/ui/__init__.py`. Either delete the empty file and restructure `ui/` as a static-files-only directory, or give the module a real purpose (e.g., template rendering helpers). Document either way.

Task 4.5 — Rebuild-reboot + commit.

Phase 5 — Coverage to 100%

Gating rule: `fail_under = 100` for each module. Raise the gate module-by-module so progress is measurable.

Task 5.1 — `download-proxy/src/api/` — lift to 100%. **Task 5.2 — `download-proxy/src/merge_service/`** — lift to 100%. **Task 5.3 — `plugins/` (canonical 12)** — lift to 100%; property-based tests for parser paths. **Task 5.4 — `plugins/community/` / `plugins/` (rest)** — at minimum smoke + contract tests per plugin. **Task 5.5 — Frontend components & services** — full Vitest suite per file listed in §A.2. **Task 5.6 — Mutation testing pass** — `mutmut run`, fix surviving mutants with targeted tests. Threshold: ≥90% mutation score.

Each task: TDD per function, commit every ≤10 tests. Pattern:

```
# Example: toast.service coverage bump
# tests/frontend/src/app/services/toast.service.spec.ts
describe('ToastService', () => {
  it('emits success() on the stream', () => { /* ... */ });
  it('auto-dismisses after timeoutMs', fakeAsync(() => { /* ...
        */ }));
  it('queue never exceeds maxVisible', () => { /* ... */ });
});
```

Phase 6 — Stress, Load, Chaos, Monitoring

Task 6.1 — Locust load profile `tests/load/locustfile.py`: 100 virtual users searching concurrently, read metrics scrape during run, assert p95 < 2s, error rate < 1%.

Task 6.2 — Stress `tests/stress/` — ramp to 1000 VU or until saturation, assert system degrades gracefully (no 5xx storm, no OOM, bounded memory).

Task 6.3 — Chaos `tests/chaos/` — toxiproxy injects latency/ packet-loss between merge service and trackers; assert circuit-breaker trips and service recovers within 60s.

Task 6.4 — Observability wiring. OpenTelemetry SDK + OTLP exporter to `otel-collector` container. Prometheus scrapes `/metrics`. Grafana dashboard JSON committed to `observability/dashboards/`.

Task 6.5 — Metric-driven tests `tests/observability/` — run load, scrape `/metrics`, assert key counters/histograms exist and advance.

Task 6.6 — Rebuild-reboot + commit.

Phase 7 — Documentation Rewrite & Extension

Task 7.1 — Missing READMEs (9 directories from §A.6). Template: purpose, entry points, conventions, tests, gotchas.

Task 7.2 — OpenAPI spec frozen. Export `/openapi.json` via `scripts/freeze-openapi.sh` into `docs/api/openapi.json` + `docs/api/openapi.md` (rendered by redoc-cli). Test: diff between live and committed spec; fail CI on drift.

Task 7.3 — Architecture diagrams. Author `docs/architecture/*.mmd` (Mermaid) for: (a) container topology, (b) request lifecycle (search → merge → stream), (c) plugin execution path, (d) private-tracker bridge flow, (e) shutdown sequence. Rendered into docs site.

Task 7.4 — "SQL definitions" clarification. The project has no relational DB. Write `docs/DATA_MODEL.md` documenting the in-memory data model (SearchMetadata, SearchResult, HookExecution) with Pydantic schemas + a Mermaid ERD-style diagram. If user later introduces SQLite/Postgres, this doc becomes the source of truth for schema migrations.

Task 7.5 — Expand `USER_MANUAL.md` with: install paths (Podman/Docker/bare-metal), every CLI flag on every script with examples, plugin install per tracker, troubleshooting matrix, FAQ, safety warnings (freeleech policy per constitution VIII).

Task 7.6 — Expand `AGENTS.md` to reflect new tooling (Snyk/Sonar/Semgrep/etc.), new test types (Part C), new compose file, new env vars.

Task 7.7 — Update `CHANGELOG.md` per semver; catch up on missed releases.

Task 7.8 — Nano-detail docs for new subsystems: `docs/OBSERVABILITY.md`, `docs/SECURITY.md`, `docs/SCANNING.md`, `docs/CONCURRENCY.md`, `docs/PERFORMANCE.md`, `docs/TESTING.md` (catalog from Part C).

Task 7.9 — Doc link-check test `tests/docs/test_no_broken_links.py` via `linkchecker`.

Phase 8 — Website (MkDocs Material)

Task 8.1 — `mkdocs.yml` at repo root, `website/` nav structure, theme Material, plugin `mkdocs-mermaid2`, plugin `mkdocs-video` (for Asciinema casts).

Task 8.2 — Landing page `website/docs/index.md`: hero, feature matrix, install quickstart, screenshots (captured from running Angular dashboard via Playwright).

Task 8.3 — Embed `docs/**` into `website/docs/` via relative imports / `mkdocs-include-markdown-plugin` so there's **one source of truth**.

Task 8.4 — GitHub Pages workflow `.github/workflows/docs.yml` builds on push to main, deploys to `gh-pages` branch.

Task 8.5 — Website build test — CI asserts `mkdocs build --strict` passes (zero broken links, zero missing refs).

Phase 9 — Video / Course Content (Asciinema + Narrated Markdown)

Task 9.1 — Directory scaffold `courses/` with subdirs: `01-operator/`, `02-plugin-author/`, `03-contributor/`, `04-security-ops/`. Each holds `script.md` + `.cast` files + thumbnail.

Task 9.2 — Record Asciinema casts for every script using `asciinema rec --command "bash -x path/to/demo.sh"`. Non-interactive — no prompts.

Task 9.3 — Embed into website via `mkdocs-video` / asciinema-player.

Task 9.4 — Test: cast files parse (`asciinema play --quiet --speed 999 <file>` exits 0).

Phase 10 — Continuous Verification & Hand-Off

Task 10.1 — Merge coverage gate to 100%. Every module. CI blocks regressions.

Task 10.2 — Rebuild-reboot — full stack, verify served content matches committed code.

Task 10.3 — Cross-check against Part A report. Each row in §A.1–A.10 has a PR link and commit SHA.

Task 10.4 — Constitution compliance review. - Principle I (Container-First): quality stack is additive compose file with justified exception → - Principle II (Plugin Contract): canonical list rationalized via §4.1 → - Principle III (Credential Security): Phase 2 fixes + Fernet-at-rest → - Principle IV (Runtime Portability): scripts still auto-detect podman/docker → - Principle V (Bridge Pattern): unchanged → - Principle VI (Validation-Driven): upgraded to automated CI → amendment needed (v1.2.0, document in `.specify/memory/constitution.md` with Sync Impact Report) - Principle VII (Operational Simplicity): no new mandatory commands; scan stack is opt-in → - Principle VIII (Freeleech): automated tests stay freeleech-only; add CI test that greps IPTorrents test for `&free=on` →

Task 10.5 — Publish release notes `CHANGELOG.md` + git tag.

Part E — Rolling Constraints (Apply To Every Phase)

- **No interactive processes.** Scripts must `set -euo pipefail`; forbidden patterns (`sudo`, `read -p`, `passwd`, `su -`) fail `tests/unit/test_scripts_non_interactive.py`.
- **No regression.** Every change guarded by pre-change test + post-change test.
- **Every commit follows** RED → GREEN → rebuild-reboot → commit (CLAUDE.md mandate).
- **Conventional commits** (`feat:`, `fix:`, `docs:`, `test:`, `chore:`, `refactor:`, `perf:`, `build:`, `ci:`).
- **Freeleech-only** for IPTorrents automated downloads (constitution VIII).
- **No `__pycache__`** committed; rebuild clears per CLAUDE.md.
- **Priority of instructions:** user prompt > CLAUDE.md > AGENTS.md > constitution > this plan > skill defaults.

Part F — Definition Of Done (whole initiative)

- [ ] Zero `pytest.skip(` related to service availability; zero `--ignore=` lines in test runner.
- [ ] `pytest --cov=download-proxy/src --cov=plugins --cov-fail-under=100` green.

- [ ] Vitest coverage ≥100% lines/branches per `frontend/vitest.config.*` threshold.
- [ ] `mutmut results` ≥ 90% killed mutants.
- [ ] Snyk + Sonar + Semgrep + Bandit + Trivy + Gitleaks: **zero high/critical** open.
- [ ] Load test p95 < 2s at 100 VU; stress test degrades gracefully at 1000 VU; chaos recovers < 60s.
- [ ] Every top-level directory has a `README.md`.
- [ ] OpenAPI frozen & diffed in CI.
- [ ] MkDocs site builds strict-mode and is deployed.
- [ ] All courses recorded and playable.
- [ ] Constitution amended to v1.2.0 with Sync Impact Report.
- [ ] Rebuild-reboot verified (served ≡ committed).
- [ ] `CHANGELOG.md` updated; release tagged.

Part G — Execution Strategy

Recommendation: Execute as **subagent-driven** (see `superpowers:subagent-driven-development`): - One fresh subagent per Task (e.g., `Task 0.1`), in a dedicated worktree. - Two-stage review: `superpowers:receiving-code-review` → integration. - Each phase is an independently mergeable PR. Phases 0–2 are serial (safety → scanners → findings). Phases 3–7 can parallelize across isolated worktrees. Phase 8–9 serial on top of 7. Phase 10 final. - Expected duration: Phase 0 (1 day), Phase 1 (1 day), Phase 2 (3–5 days, backlog-driven), Phase 3 (3 days), Phase 4 (2 days), Phase 5 (5–10 days), Phase 6 (3 days), Phase 7 (3 days), Phase 8 (1 day), Phase 9 (2 days), Phase 10 (1 day). **Total: ~3–5 weeks of focused work.**

Alternative: Inline execution via `superpowers:executing-plans` — slower iteration, single session.

Open Questions For User (blocks Phase 8 & 9)

1. **Website**: MkDocs Material (recommended), Astro, or defer?
2. **Video courses**: Asciinema + narrated markdown (recommended), scripts+storyboards for external recording, or defer?
3. **Plugin dead code**: for the 36 non-canonical plugins, default is "move to `plugins/community/` with smoke tests". Confirm, or specify per-plugin actions?
4. **Constitution amendment**: Phase 10 proposes v1.2.0 to formalize CI as a hard gate. Approve?
5. **Observability (Phase 6)**: Prometheus+Grafana+OTel as opt-in compose. Confirm opt-in vs mandatory?